

UNIVERSITY OF THE ARMED FORCES - ESPE

Computer Science Department

Object Oriented Programming

36863

Homework05 - Slides Summary

UML Reading

Dennis Alejandro Noboa Montesdeoca

LASCANO, JORGE EDISON

October 14, 2026

13	**Last Name:** Noboa Montesdeoca	**First Name:** Dennis Alejandro	**Grade:**

SLIDES SUMMARY 14 10 26

1. OBJECT-ORIENTED MODELING PRACTICES

CONCEPTUAL MODELING OVERVIEW

- Purpose: Conceptual modeling allows software engineers to capturate, analyze, prioritize, and communicate ideas and design decisions before and during system construction.
- Goals: It helps developers understand real-world problem domains, visualize existing systems, specify software architectures, and build actionable implementation plans.
- Perspectives: Modeling occurs across all stages of software development including Analysis, Design, Specification, Implementation, and Deployment.

CORE OBJECT-ORIENTED PRINCIPLES

- Layered Architecture: Systems should be organized into loosely coupled layers (e.g., User Interface, Application Logic, Persistence, Communications, Operating System). Each layer provides clean abstractions to higher layers while depending only on a components in its own or lower layers.
- Prefer Aggregation Over Inheritance: Aggregation (e.g., class A contains class B) provides greater runtime flexibility than static class inheritance. Inheritance tightly couples classes at compile time, whereas aggregated relationships can be dynamically resigned during execution.
- Program to an Interface or Abstraction: Decouple system components by interacting through abstract classes or class interfaces rather than concrete implementations.
- Naming Conventions:
  - Classes: Use singular nouns/noun phrases in PascalCase (UpperCamelCase) reflecting a prototypical instance (e.g., Student, WidgetB).
  - Variables: Use lowerCamelCase representing the purpose or role of the held object (e.g., myWidgetB).
  - Methods/Operations: Use action verbs or verb phrases in lowerCamelCase (e.g., computeCost, displayI).

17 10 26

- Executable Unit Testing: Developers should write automated unit test cases to detect design errors early, verify correctness repeatably, document expected usage, and maintain regression safety.

2. INTRODUCTION TO UML (UNIFIED MODELING LANGUAGE)

NATURE & BUILDING BLOCKS OF UML

- Definition: UML is an expressive, semi-formal, process-independent, and extensible modeling visual language.
- Building Blocks:
  - Modeling Elements: Conceptual items categorized into structural (e.g., classes, objects, interfaces, components, nodes), Behavioral (e.g., messages, states, events), organizational (e.g., packages, views), and Annotational (e.g., comments, notes).
  - Diagrams: Visual groupings of modeling elements organized like sentences / paragraphs.
  - Views & Models: Collections of diagrams describing a complete system from specific perspective.

THREE SOFTWARE ENGINEERING PERSPECTIVES

1. Analysis Perspective: Focuses on understanding the problem domain using real-world concepts to describe systems as they currently exist.
2. SPECIFICATION / High-Level Design Perspective: Combines real-world and software abstractions to establish system interfaces, logical responsibilities, and automation boundaries.
3. Implementation / Design Perspective: Focuses on concrete solution-oriented software components (e.g., data structs, GUI widgets, framework methods, memory layout).

CLASSIFICATION OF UML DIAGRAMS

- Structural Diagrams (Static Architecture):
  - Class Diagrams: Model classes, interfaces, collaborations, and structural relationships.
  - Object Diagrams: Model snapshots of instantiated objects and their attribute values at runtime.

Scanned with CamScanner
Scanned with CamScanner

DD MM AA
14 10 26

- Component Diagrams: Show physical implementation models, software components, and their dependencies.
- Deployment Diagrams: Show hardware execution nodes and the distribution of software components across nodes.

- Behavioral Diagrams (Dynamic Execution):
  - Use Case Diagrams: Capture actors, goals, and system boundaries.
  - Sequence Diagrams: Highlight the temporal (chronological) ordering of messages between objects.
  - Collaboration Diagrams: Highlight structural relationships among communicating objects.
  - Statechart Diagrams: Model object lifecycle, states, events, and state transitions.
  - Activity Diagrams: Represent control flows, execution steps, and parallel branches within system processes.

3. Use Case Modeling

Use case modeling documents the functional requirements of a system from the perspective of the end user or external systems. It captures what the system should do and why, independent of design or implementation decisions.

Core Components

1. Actors:
   - Definition: External entities outside the system that interact with it to achieve a specific goal.
   - Types: Human users (roles), other software systems, or hardware devices/sensors.
   - Notation: Named in the singular representing a role (e.g., Teacher, Student) and represented visually by a stick figure.

DD MM AA
14 10 26

2. Use Cases:
   - Definition: Represent discrete, user-visible goals and functionality needed from the system.
   - Naming: Short names expressed in terms that are free of design choices (e.g., Record class grades, Review transcript).

3. Subject / System Boundary:
   - A structural rectangle enclosing use cases to define ownership and clarify what functionality belongs inside versus outside the system boundary.

Use Cases vs. System Interactions

- Use Cases: Answer "What" and "Why" from an abstract user-goal perspective.
- System Interactions: Answer "How" by detailing step-by-step user interface actions (e.g., Select semester → Select course → Review grade → Enter new grade → Confirm). Confusing system interactions with use cases limits design creativity and obscures core system purpose.

Relationships in Use Case Diagrams

- Generalization:
  - Defines sub-type/sub-type between actor or use cases where a specialized element inherits features and refines a general one.
  - Symbol: Solid line with a hollow triangular arrowhead pointing to the general element.
- <<include>> Dependency:
  - Defines how a base use case explicitly requires the completion of another use case's goal to fulfill its own (a form of goal aggregation).
  - Symbol: Dashed line with an open arrowhead pointing to the included use case, labeled <<include>>.

Scanned with CamScanner
Scanned with CamScanner

DD MM AA
17 10 26

- <<extend>> Dependency:
  - Defines optional, conditional variations or augmentations that extend a base use case under specific conditions at designated Extension Points.
  - The extended base use case can stand on its own without the extension.
  - Symbol: Dashed line with an open arrowhead pointing from the extending use case to the base use case, labeled <<extend>>.

4. Objects, Classes, and Class Diagrams.

Class diagrams describe the static structural architecture of a system by modeling object types, internal properties, and relationships.

Fundamental Concepts

- Objects:
  - Fundamental building blocks of object-oriented systems.
  - Encapsulate state (attribute values), Behavior (operations), and a unique identity.
  - Clarification: Transient events, pure behaviors, constraints, and raw relationships are generally not modeled as individual objects.
- Classes:
  - Abstractions that classify collections of objects sharing common attributes, operations, and semantic behaviors.

Class Syntax and Structure in UML

A class is rendered as a rectangle divided into vertical compartments.

1. Class Name Compartment:
   - Singular noun or noun phrase capitalized in UpperCamelCase (e.g., Student, Course).

DD MM AA
17 10 26

2. Attributes Compartment:
   - Placeholders holding state values.
   - Full syntax:
     [visibility] name [multiplicity] [: type] [= initial-value]
     [{ property-string }]
   - Visibilities: Public (+), Protected (#), Private (-).
   - Example: -gpa: Real = 4.0
3. Operations / Methods Compartment:
   - Functions or method declarations providing services.
   - Full syntax:
     [visibility] name [(parameter-list)] [: return-type]
     [{ property-string }]
   - Example: + addSection(section: ClassSection): Boolean
4. Responsibilities & Notes Compartment (Optional):
   - Textual notes stating class responsibilities, exceptions, or business rules.

Object Syntax in Object Diagrams

- Represents static snapshots of instantiated runtime objects.
- Object names are underlined using the format instanceName: ClassName (e.g., joe: Student).
- An omitted instance name represents an anonymous instance (e.g., : Student).
- Attributes inside object boxes show assigned runtime values (e.g., major = "CS", gpa = 3.01).

Class Relationship Types

1. Generalization (Inheritance):
   - An "is-a" sub-typing relationship where a child class inherits attributes, operations, and relationships

Scanned with CamScanner
Scanned with CamScanner

DD MM AA
14 10 26

from a superclass.

- Symbol: Solid line with a hollow triangular arrowhead pointing to the superclass.

2. Association:

- Structural link specifying that instances of one class connect to instances of another.

- Can be named (lowercase verb with a directional arrow >), uni-directional, bi-directional, or n-ary (connecting more than two classes)

- Symbol: Solid line between classes.

3. Multiplicity Constraints:

- Quantifies how many instances can link across an association.

- Syntax: 1 (exactly one), 0..1 (zero or one), * or 0..* (many), 1..* (one or more), 1..3 (specific range).

4. Aggregation:

- A "part-of" association.

- Represents weak composition: subpart instances can exist independently of the aggregate parent.

- Symbol: Solid line with an open / hollow diamond at the container end.

5. Composition:

- A strong form of aggregation where subpart instances lifecycles are strictly bound to the aggregate parent instance.

- Deleting the parent automatically deletes the contained subparts.

- Symbol: Solid line with a solid/ black diamond at the container end.

Scanned with CamScanner

14 10 26

6. Dependency:

- A "use" relationship indicating that changes to one class may require updates in the dependent class.

- Common when a class receives another class as a method parameter or instantiates it locally without storing a permanent link.

- Symbol: Dashed line with an open arrowhead pointing to the class being depended upon.

Norma

Scanned with CamScanner